UNIVERSITY OF OULU

FACULTY OF INFORMATION TECHNOLOGY AND ELECTRICAL ENGINEERING

Syed Abdullah Hassan

DISTRIBUTED MEMORY BUS FOR MULTI-AGENT CAMPUS SYSTEMS: CONTEXT COMPRESSION, COORDINATION QUALITY, AND POLICY-AWARE SHARING

Master's thesis
Degree Programme in Computer Science and Engineering
May 2026

Hassan S. (2026) Distributed Memory Bus for Multi-Agent Campus Systems: Context Compression, Coordination Quality, and Policy-Aware Sharing. University of Oulu, Faculty of Information Technology and Electrical Engineering, Degree Programme in Computer Science and Engineering, Master's thesis, 43 p.

ABSTRACT

This thesis designs, implements, and evaluates a distributed memory bus with a context-compression layer for multi-agent institutional systems. It contributes (i) a multi-agent coordination benchmark, (ii) an empirical characterisation of how context compression affects multi-agent coordination quality at 7B parameters with a 13B sanity arm, (iii) a catalogue of three retrieval-augmented generation pipeline architectures combining retrieval and compression under matched evaluation conditions, and (iv) a tag-preserving compressor variant integrated with a reference memory-bus implementation. The compute envelope is a single Apple M4 Pro 48 GB workstation, and the deliverable is the manuscript together with an open-source reference implementation.

Multi-agent systems built on large language models share their working state through a context window that is fundamentally bounded. As institutional deployments scale across many cooperating agents, the cost of moving that state grows fast enough to dominate operating expenses, and the temptation to compress it is correspondingly strong. The empirical question this thesis asks is whether single-agent metrics — accuracy on a question-answering task, faithfulness on a retrieval task — continue to predict the quality of *coordination* between agents under compression, or whether compression introduces a regime in which coordination breaks faster than single-agent quality does.

The thesis contributes four artefacts. First, a multi-agent coordination benchmark inspired by a period-end reporting workflow that spans eight university systems, with three workload families and coordination-quality metrics that go beyond per-turn correctness. Second, an empirical characterisation of three compressor families — LLMLingua-2, an instruction-aware heuristic filter, and an ICAE-style soft-prompt compressor — across compression ratios from $1\times$ to $16\times$ at 7B parameters, with a secondary 13B sanity arm, centred on the existence and shape of a *coordination cliff*: a compression ratio beyond which coordination success degrades sharply even when single-agent QA accuracy degrades smoothly. Third, a catalogue of three retrieval-augmented generation pipelines combining retrieval and compression in different orders, benchmarked under matched retrieval quality with a cost model in EUR per query. Fourth, a tag-preserving compressor variant that lets provenance and access-control metadata travel alongside compressed memory slots, evaluated on preservation rate and accuracy delta. All four contributions are delivered with a single-machine reference implementation built on a 48 GB Apple Silicon workstation and an open-source repository with one-command reproduction of every headline figure.

Keywords: multi-agent systems, large language models, context compression, retrieval-augmented generation, agentic memory, privacy-aware retrieval

Hassan S. (2026) Hajautettu muistiväylä monitoimijajärjestelmille kampusympäristössä: kontekstin tiivistäminen, koordinaation laatu ja käytäntötietoinen jakaminen. Oulun yliopisto, Tieto- ja sähkötekniikan tiedekunta, Tietotekniikan tutkinto-ohjelma, Diplomityö, 43 s.

TIIVISTELMÄ

Tämä diplomityö suunnittelee, toteuttaa ja arvioi hajautetun muistiväylän kontekstin tiivistyskerroksella monitoimijallisille instituutiojärjestelmille. Työn kontribuutiot ovat (i) monitoimijallinen koordinaatio-benchmark, (ii) empiirinen karakterisointi siitä, kuinka kontekstin tiivistys vaikuttaa monitoimijoiden koordinaation laatuun 7B-mallikoossa ja 13B-tarkistusvarsi, (iii) kolmen RAG-arkkitehtuurin luettelo, joissa haku ja tiivistys yhdistetään yhdenmukaistettujen arviointiehtojen alla, sekä (iv) merkintöjä säilyttävä tiivistäjävariantti, joka on integroitu viiteimplementaatioon. Laskentakapasiteetti rajoittuu yhteen Apple M4 Pro 48 GB -työasemaan ja toimitettava kokonaisuus koostuu käsikirjoituksesta ja avoimen lähdekoodin viiteimplementaatiosta.

Avainsanat: monitoimijajärjestelmät, suuret kielimallit, kontekstin tiivistäminen, hakuparannettu generointi, agenttinen muisti, yksityisyystietoinen haku

TABLE OF CONTENTS

FOREWORD

This thesis was carried out at the Future Computing Group, CSE Research Unit, Faculty of Information Technology and Electrical Engineering, University of Oulu, in collaboration with TalentAdore Ltd. The work was supervised academically by Lauri Lovén, with industrial supervision from Asim Nadeem and Oskari Valkama at TalentAdore.

I owe my warmest thanks to Lauri for the latitude to scope a project around an unfamiliar empirical question — the way coordination under compression behaves at institutional scale — and for the discipline he brought to keeping the scope tight. I am grateful to Asim Nadeem and Oskari Valkama for sharing TalentAdore's production cost structure, which anchored the cost-per-workflow arm of the evaluation in a real industrial setting. I thank Mohammad Abaeiani, Faisal Khan, and Vu Truong for their willingness to coordinate around the demands of this thesis.

The thesis was prepared on a single Apple M4 Pro 48 GB workstation. All training, evaluation, and inference fit within that envelope. The reproducibility package and the open-source reference implementation accompany this manuscript.

Oulu, 18th May, 2026

Syed Abdullah Hassan

LIST OF ABBREVIATIONS AND SYMBOLS

ACL	access control list
ADR	architecture decision record
API	application programming interface
BCa	bias-corrected and accelerated (bootstrap CI)
CI	confidence interval
C1 . . . C4	contributions one through four of this thesis
CSE	Computer Science and Engineering
DAG	directed acyclic graph
EM	exact match (QA metric)
FAISS	Facebook AI Similarity Search
FCG	Future Computing Group
fp16	16-bit floating-point
GGUF	GPT-Generated Unified Format (llama.cpp model file format)
H1 . . . H4	hypotheses one through four
HNSW	hierarchical navigable small world (vector index)
HTTP	hypertext transfer protocol
int4	4-bit integer quantisation
ICAE	in-context autoencoder
ITEE	Information Technology and Electrical Engineering
KV cache	key–value cache (transformer attention)
LCS	longest common subsequence
LLM	large language model
LoRA	low-rank adaptation
LRU	least recently used
M0 . . . M5	sibling thesis projects (Publication DB, Contract DB, Moodle/Patio, etc.)
MLX	Apple's native ML framework for Apple Silicon
MPS	Metal Performance Shaders (Apple PyTorch backend)
NCE	noise-contrastive estimation
NIST AI RMF	NIST AI Risk Management Framework
OAuth	open authorisation protocol
OTel	OpenTelemetry
P1 . . . P3	retrieval–compression pipeline variants one through three
PEFT	parameter-efficient fine-tuning
PII	personally identifiable information
QA	question answering
RAG	retrieval-augmented generation
SQL	structured query language
SQLite	a lightweight, embedded SQL database
SSE	server-sent events
TTL	time-to-live
UUID	universally unique identifier
WAL	write-ahead log

L_{recon}	token-level reconstruction loss
L_{NCE}	InfoNCE contrastive loss
L_{tag}	per-slot tag-prediction loss (C4 only)
λ	loss-term weighting coefficient
M	number of memory slots in the soft-prompt compressor
ρ	Spearman's rank correlation coefficient
τ^*	coordination cliff compression ratio
r	nominal compression ratio (input/output token count)
δ	Cliff's delta (ordinal effect size)
d	Cohen's d (continuous effect size)

1. INTRODUCTION

1.1. Motivation

Large language models (LLMs) have moved from single-turn assistants to multi-agent systems in which a planner, several workers, and a critic cooperate on a shared task by reading and writing a common context [1–4]. The shared context is finite. As institutions deploy these systems at scale, the cost of moving state between agents grows fast enough to dominate operating expenses [5, 6]. The closest published industry analogue, Anthropic's research team, reports that token usage alone explains approximately eighty percent of the performance variance in their multi-agent research system [4]. Their context editing and memory-tool announcement reports an eighty-four percent token reduction on a 100-turn web-search evaluation, which is the practical setting in which compression is no longer optional but unavoidable [7].

Context compression as a research area is well developed in the single-agent setting. LLMLingua and its successors compress prompts with small classifier networks that drop low-utility tokens [8–10]. Soft-prompt approaches such as gist tokens, AutoCompressor, and the In-Context Autoencoder (ICAE) train a small adapter to summarise a passage into a fixed set of memory slots [11–14]. Surveys consolidate the design space and the reported accuracy–ratio trade-offs [15, 16].

What the literature does not address directly is whether the single-agent accuracy curve continues to predict the quality of *coordination* between agents under the same compression. A multi-agent task is not just a sum of single-agent QA accuracies; the planner has to assign sub-tasks based on a compressed view of the evidence, workers have to retrieve based on a compressed scratchpad, and the critic has to verify a draft answer against compressed source fragments. There is no shared empirical characterisation of how compression degrades that pipeline.

The Future Computing Group (FCG) at the University of Oulu is building an institutional AI service economy that puts compression on the critical path of every multi-system workflow [17–19]. The flagship use case, *Vignette 3.7 – Period-end project reporting*, gathers reporting data across eight university systems via roughly eight specialised agents, achieves an estimated 80% cloud-payload reduction, and lowers the per-report cost to about EUR 2.15 [20]. TalentAdore, the industry partner of this thesis, runs production workloads whose operating expenses are dominated by token cost; their interest is the cost-per-workflow story that compression makes possible. The master's-scale problem this thesis carves out is the question of whether compression preserves *coordination*, and at what point it breaks.

1.2. Thesis Statement and Contributions

This thesis designs, implements, and evaluates a distributed memory bus with a context-compression layer for multi-agent institutional systems. The empirical question is whether single-agent QA accuracy is a faithful predictor of multi-agent coordination success under compression, and the methodological question is whether such a coordination metric can be defined and measured.

The thesis contributes four artefacts, corresponding to the four research questions accepted at the project's inception:

1. **C1 — A multi-agent coordination benchmark.** A reproducible synthetic benchmark inspired by Vignette 3.7: a planner, a configurable number of workers, and a critic exchanging compressed state through a shared scratchpad. Three workload families, configurable agent count, source-document characteristics, tag distributions, and coordination-quality metrics that go beyond per-turn correctness (planner-replan count, sub-task-assignment accuracy, rounds-to-completion, critic-flagged error rate). Released alongside the thesis with documentation and seed configurations.
2. **C2 — Empirical characterisation of compression effects on multi-agent coordination.** Three compressor families — LLMLingua-2 as a token-level hard-prompt filter [10], an instruction-aware heuristic filter, and an ICAE-style soft-prompt compressor that is the trainable arm [12, 13] — evaluated on the C1 benchmark at compression ratios from 1× to 16× at 7B, with a secondary 13B sanity arm. The headline empirical result is the existence and shape of a *coordination cliff* τ^*: a compression ratio beyond which coordination success degrades sharply even when single-agent QA accuracy degrades smoothly.
3. **C3 — Joint RAG plus compression pipeline catalogue.** Three pipeline architectures implemented end-to-end on FAISS and LlamaIndex [21, 22]: P1 (compress, then retrieve), P2 (retrieve, then compress; the classical LongLLMLingua setup [9]), and P3 (joint, conditional compression based on retrieval relevance scores, following the structure of [23]). Each pipeline is benchmarked under matched retrieval quality, with a cost model in EUR per query fitted to TalentAdore production pricing for GPT-4o-mini and Claude Haiku 4.5 plus amortised local inference cost on the M4 Pro.
4. **C4 — Tag-preserving compressor variant and reference memory-bus integration.** A modification of the ICAE-style compressor that adds a per-slot tag-prediction head emitting an inherited provenance and access-control-list vector for each compressed fragment [24, 25]. Evaluated on tag-preservation rate at ratios $\{2\times, 4\times, 8\times\}$ and on accuracy delta relative to the unmodified variant. Delivered with a reference memory-bus integration that exposes the read, write, subscribe, and audit interfaces.

The four contributions form a sequence: build the testbed, use it to make a measurement the field has not made, embed compression into the RAG pipelines institutional agents actually use, and then probe what happens when compression carries metadata.

1.3. Scope and Compute Envelope

The thesis runs on a single Apple M4 Pro 48 GB workstation using MLX for training and 7B/13B inference, Ollama for developer-experience inference, HuggingFace Transformers with PEFT for fine-tuning, FAISS-CPU for vector search, LlamaIndex for RAG orchestration, AutoGen v0.4 for agent runtime, and SQLite for the audit log [1, 21, 22, 26–29]. Every wallclock estimate in the thesis is dimensioned for that envelope.

Three items are explicitly out of scope: production-grade governance enforcement (the tag-preserving compressor is a research prototype with measured properties, not a compliance tool); cross-tokenizer or cross-model-family compressed exchange; and a live production deployment on the FCG platform (the integration interface is delivered as a reference implementation that the FCG team can adopt).

1.4. Hypotheses

Four hypotheses guide the empirical work; each is grounded in a specific measurable comparison with an explicit falsifiable claim. All four are thesis hypotheses, and falsification of any one is itself reported as a thesis-worthy finding in the relevant chapter. The hypotheses are summarised in Chapter Chapter 4 and listed here only for completeness:

- H1. Single-agent QA accuracy under compression is a poor predictor of multi-agent coordination success.
- H2. A coordination cliff τ^* exists and is task-dependent. A secondary 13B arm checks whether τ^* shifts with model size.
- H3. RAG pipeline placement matters: P1 and P2 dominate in different regimes, and P3 closes the gap on a combined accuracy-times-cost score.
- H4. A per-slot tag head preserves provenance tags at rate at least 85% at 4× compression with accuracy degradation at most 5 pp.

The narrative arc is: standard metrics are incomplete (H1), so a coordination cliff exists and is measurable (H2); RAG pipeline placement matters (H3); and governance metadata can ride along compression (H4).

1.5. Thesis Structure

Chapter Chapter 2 reviews related work in five strands: context compression and memory, multi-agent systems and agentic memory, RAG and long-context benchmarks, privacy-aware retrieval and adversarial robustness, and the FCG programme and its industry-relevance arm. Chapter Chapter 3 describes the system design and implementation: the three-layer memory-bus architecture, the compressor framework with its four variants, the C1 benchmark generator, the RAG pipeline catalogue, the agent runtime, and the inference-backend stack. Chapter Chapter 4 specifies the experimental protocol for each of the four hypotheses, the metrics, and the statistical analysis plan, with placeholder result sections marked clearly. The manuscript closes with Chapter Chapter 5, which discusses the limitations and the deliverables.

1.6. Author's Contributions and the Role of Artificial Intelligence

The implementation, experimental design, training procedures, statistical protocol, and writing of this thesis are the independent work of the author, conducted under the supervision arrangement described in the foreword. Generative AI tools (Claude and ChatGPT) were used as writing aids in three documented capacities: as a literature-review assistant for locating and cross-referencing prior work, as a code-review assistant for catching defects in the reference implementation, and as a copy-editor for clarity and grammar in the manuscript. All technical decisions, architectural choices, experimental designs, statistical methods, hypothesis formulations, and conclusions are the author's own. No AI system produced any results or empirical claims in the thesis; every empirical result reported (when populated) is generated by the reproducible scripts in the open-source repository accompanying the manuscript.

2. BACKGROUND AND RELATED WORK

This chapter reviews prior work that grounds the four contributions of the thesis. It is organised into five strands. Section 2.1 covers context compression and agentic memory; Section 2.2 covers multi-agent systems and the closest published industry analogue; Section 2.3 covers retrieval-augmented generation, long-context benchmarks, and the evaluation suite that frames C3; Section 2.4 covers privacy-aware retrieval; and Section 2.5 situates the work within the Future Computing Group programme at the University of Oulu and the TalentAdore industry use case.

2.1. Context Compression and Agentic Memory

2.1.1. Hard-Prompt Token Filtering

The first generation of practical context-compression techniques operated at the token level. Jiang *et al.* [8] introduced LLMLingua, which uses a small autoregressive language model to score tokens by their per-step perplexity contribution and drop those whose removal least disturbs the conditional distribution. The same line of work produced LongLLMLingua [9], which extends the coarse-to-fine ranking with a question-aware step to handle the retrieval setting, and LLMLingua-2 [10], which trains an XLM-RoBERTa token classifier on data distilled from a more capable model, yielding a task-agnostic compressor. Because the underlying XLM-RoBERTa model is supported by PyTorch on both CPU and Apple's MPS backend, the compressor runs natively on Apple Silicon without modification. Pan *et al.* report compression ratios up to 5× with less than two percentage points of F1 loss on MeetingBank summarisation and GSM8K reasoning tasks, and their data-distillation approach eliminates the need for question-aware coarse-to-fine ranking at inference time, making the compressor substantially faster than the original LLMLingua.

This thesis takes LLMLingua-2 as the principal hard-prompt baseline. The model is bundled with the official `llmlingua` package, which makes it the de facto baseline against which any new compressor must be calibrated. The thesis includes a stop-the-line criterion that NarrativeQA F1 must reproduce within 2 pp and HotpotQA F1 within 3 pp of reported numbers [30, 31].

2.1.2. Soft-Prompt Compression

A parallel line of work compresses passages into a small number of continuous embedding slots that occupy the same positions in a frozen decoder's attention windows. Mu *et al.* [11] introduced *gist tokens*, special positions trained with a masked attention pattern so that the model learns to compress instruction prefixes into a small number of virtual tokens, reducing prompt length by up to 26× with negligible quality loss on instruction-following tasks. Chevalier *et al.* [12] extended this idea with *AutoCompressor*, which fine-tunes a language model to process long documents segment by segment and produce *summary vectors* at each segment boundary; the summary vectors are prepended to the next segment's input, creating a recurrent compression mechanism trained with an unsupervised next-token prediction loss over the segmented document. Their demonstration that summary vectors can carry sufficient information to support downstream

language modelling across segment boundaries is the design principle this thesis builds on; the ICAE architecture below adapts it by adding LoRA to the encoder and freezing the decoder.

Ge *et al.* [13] formalised this architecture as the *In-context Autoencoder (ICAE)*, which trains a LoRA-adapted encoder to produce a fixed number of memory tokens from an input context; these memory tokens, when prepended to the frozen decoder's input, allow it to reconstruct the original context or answer questions about it. The ICAE achieves 4× compression with minor accuracy degradation on NaturalQuestions and other reading-comprehension benchmarks. Wang *et al.* [14] present the In-Context Former, a faster cross- attention variant that drops the reconstruction objective in favour of direct task prediction. Fei *et al.* [32] compress at the semantic-unit level rather than the token level, preserving sentence boundaries and discourse markers that token-level methods may discard. Rae *et al.* [16] establish that compressed memory in transformers has a degradation curve that is smooth and monotonic in the single-context regime — a baseline that the coordination-cliff hypothesis (H2) explicitly tests against in the multi-agent setting.

The trainable arm of this thesis is an ICAE-style compressor with the dual-objective loss documented in Section 3.3.6 of Chapter Chapter 3. The construction follows Ge *et al.* [13] for the architecture and Chevalier *et al.* [12] for the segment-level language-modelling objective. The contrastive term is an InfoNCE loss [33, 34] sampled over positive–negative source-fragment pairs.

2.1.3. Compression for RAG

A third strand of work pushes compression into the retrieval-augmented generation pipeline. Cheng *et al.* [35] propose *xRAG*, which compresses retrieved chunks to a single soft-prompt token, anchoring the extreme end of the ratio spectrum. Guo *et al.* [23] make compression *relevance-conditional*: chunks whose retrieval scores exceed a threshold are kept verbatim, those below are compressed, and those well below are discarded. The third pipeline of this thesis (P3) follows their structural design directly, while the first two pipelines (P1 and P2) reproduce the classical *compress–then–retrieve* and *retrieve–then–compress* configurations against which P3 must demonstrate a benefit. Li *et al.* [15] publish a survey of prompt compression methods that consolidates this design space and provides the taxonomic vocabulary used throughout this thesis.

2.1.4. LLM-As-Operating-System Memory Hierarchies

Packer *et al.* [36] introduced *MemGPT*, which models the LLM as an operating system that pages information between a fast in-context working set and a slow archive, using function calls to manage the boundary between main context and external storage. Xu *et al.* [37] propose *A-MEM*, which extends this with Zettelkasten-style linking between memory entries, creating a structured knowledge base from the agent's past observations. Chhikara *et al.* [38] and Rasmussen *et al.* [39] target the production end of the spectrum with embedding-based extraction (*Mem0*) and temporal knowledge-graph memory (*Zep*). Yu *et al.* [40] learn to retrieve from past actions via reinforcement learning. The memory bus in this thesis is not a replacement for any of these but a substrate on top of which they could sit: the storage layer protocols are

factored so that an OS-style pager, a knowledge graph, or an episodic store all conform to the same contract.

2.2. Multi-Agent Systems and Coordination

2.2.1. Multi-Agent Frameworks

Wu *et al.* [1] introduced *AutoGen*, a conversational multi-agent framework with role-based agents and tool use that became the de facto research vehicle for studying planner-worker-critic topologies. The v0.4 rewrite introduced an async event-driven core and the `RoundRobinGroupChat` primitive that this thesis uses as the skeleton for its planner-worker-critic loop. Hong *et al.* [2] introduced *MetaGPT*, which encodes standardised operating procedures (SOPs) as structured artefacts passed between agents; the SOP pattern informs the sub-task assignment protocol of the planner in this thesis. Li *et al.* [41] explored role-playing dialogues for autonomous agents (*CAMEL*). Shinn *et al.* [3] introduced *Reflexion*, which formalises the critic loop as verbal reinforcement learning — a self-reflection mechanism where the agent generates linguistic feedback on its own outputs and uses that feedback to improve subsequent attempts. The critic agent in this thesis emits a structured `DONE` or `REVISE` verdict in the spirit of Reflexion.

2.2.2. Token-Efficient Multi-Agent Coordination

A small but growing literature directly attacks the multi-agent token-efficiency problem. Zhang *et al.* [42] introduce *AgentPrune*, which sparsifies the communication graph between agents by learning which inter-agent messages can be dropped without degrading task performance. Wang *et al.* [43] propose *AgentDropout*, which stochastically drops entire agents during multi-agent inference, discovering that many agents in over-provisioned teams contribute redundant computation. Ye *et al.* [44] introduce *KVCOMM*, which shares KV-cache prefixes across agents to amortise the communication cost of transmitting large context windows between cooperating LLMs.

Park *et al.* [24] introduce *Collaborative Memory*, which pushes per-fragment access-control metadata into a shared memory layer for multi-user agent systems. Their work demonstrates that dynamic access control can be implemented at the memory level rather than at the application boundary, enabling agents to selectively share information based on trust levels. This is the closest published precedent for the C4 tag-preserving extension and the structural ancestor of the per-slot tag head in Section 3.3.5 of Chapter Chapter 3.

2.2.3. The Closest Published Industry Analogue

Anthropic's published descriptions of their multi-agent research system provide the closest industry analogue to the empirical question of this thesis. Their engineering blog [4] describes an orchestrator-worker topology for multi-agent research in which *token usage* is identified as the dominant factor in system performance, and their context-engineering write-up [6] discusses techniques for managing context in long-horizon agentic tasks, including *compaction* (of which

tool-result clearing is the lightest form), *structured note-taking* as a form of agentic memory, and *sub-agent architectures*, alongside a separate treatment of *just-in-time retrieval* for context loading. These techniques frame the design choices in C3 (RAG pipeline placement). Their announcement of context editing and the memory tool on the Claude Developer Platform [7] reports significant token reductions via context editing on multi-turn evaluation tasks. These results are the strongest signal in the public literature that aggressive context management materially affects multi-agent performance; H2 in this thesis is the first attempt the author is aware of to characterise the shape of that degradation empirically as a function of compression ratio.

2.2.4. Context Engineering for Multi-Agent Assistants

Haseeb [45] studies context engineering in multi-agent code assistants, finding that the distribution of training data (monolithic QA versus multi-turn dialogue traces) affects coordination quality independently of single-agent accuracy. The stretch goal S3 (Chapter Chapter 5) would implement his approach as a baseline comparator on C1.

2.3. Retrieval-Augmented Generation and Long-Context Benchmarks

2.3.1. Retrieval-Augmented Generation

Lewis *et al.* [46] introduced the original retrieval-augmented generation framework, which combines a parametric generator with a non-parametric retrieval index and demonstrated state-of-the-art results on open-domain question answering. The descendant literature has explored hierarchical indices that recursively summarise document clusters [47], graph-structured summaries that capture entity relationships across documents [48], neurobiologically inspired memory organisation that mirrors hippocampal indexing [49, 50], and self-reflective gating that decides on the fly whether to retrieve at all based on the model's confidence in its own parametric knowledge [51]. Anthropic's contextual-retrieval write-up [52] reports that prepending model-generated chunk-level context to each document chunk before embedding improves retrieval quality across codebases, fiction, arXiv preprints, and scientific papers — a useful pre-processing comparator for P1 in the C3 catalogue.

The C3 catalogue of this thesis (P1, P2, P3) sits at the intersection of retrieval and compression. P1 is the *compress-then-retrieve* configuration, where the index stores compressed chunks; P2 is the *retrieve-then-compress* configuration that recovers the LongLLMLingua setup [9]; P3 is the *joint* configuration that routes chunks through one of three branches based on retrieval relevance, following the structural design of Guo *et al.* [23].

2.3.2. Long-Context Benchmarks

Liu *et al.* [53] document the *lost-in-the-middle* phenomenon: language models attend disproportionately to the beginning and end of their context windows, with a U-shaped attention pattern that causes information placed in the middle of long contexts to be disproportionately ignored. This finding motivates compression as a strategy not just for cost reduction but for

quality improvement — by compressing away padding and low-relevance material, the remaining content occupies positions where the model's attention is stronger. Bai *et al.* [54] maintain LongBench, a suite of long-context tasks covering single-document QA, multi-document QA, summarisation, few-shot learning, synthetic retrieval, and code completion, that this thesis uses as an external calibration anchor for the compressor baselines.

2.3.3. Agent-Task Benchmarks

GAIA [55] and AgentBench [56] motivate the agent-task framing of this thesis. GAIA provides multi-step reasoning tasks at three difficulty levels that require an agent to use tools, browse the web, and synthesise information across sources. AgentBench evaluates agents across diverse environments including operating systems, databases, and web browsing. The C1 benchmark is *not* a competitor to either; it isolates the specific question of coordination-quality-versus-compression, which neither GAIA nor AgentBench poses directly.

2.4. Privacy-Aware Retrieval

The tag-preserving extension (C4) is grounded in a small but active literature on privacy-aware retrieval. Zhou *et al.* [57] propose a privacy-aware variant of RAG that filters retrieved documents against requester-side policy before they reach the generator, preventing the model from conditioning on information the requester is not authorised to see. Bassit and Boddeti [58] push the privacy guarantee deeper, providing cryptographic guarantees on the retrieval side so that even the retrieval index cannot learn which documents a requester accessed.

Li *et al.* [59] introduce *SecurityLingua*, a red-team treatment of prompt compression that constructs adversarial prompts specifically designed to exploit the token-dropping behaviour of compressors. Their demonstration that compressed prompts can be manipulated to bypass safety filters motivates the S2 stretch goal (Chapter Chapter 5), which would run a SecurityLingua-style sweep against the tag-preserving variant.

The NIST AI Risk Management Framework [25] is cited as the lattice from which the 5-tier classification hierarchy (PUBLIC < INTERNAL < CONFIDENTIAL < RESTRICTED < SECRET) used by the tag vector in Section 3.2 of Chapter Chapter 3 is derived. This thesis does not claim NIST compliance; the framework provides a vocabulary for the classification tiers, not a certification target.

2.5. The FCG Programme and the Industry Setting

The thesis sits within the Future Computing Group's institutional AI service economy research programme at the University of Oulu. The programme envisions a marketplace in which AI service providers (institutional LLM deployments, cloud APIs, on-premises inference clusters) compete to serve agent requests, with a neutral exchange earning per-unit fees on volume [60]. Three internal architecture documents specify the target platform: the system architecture [17], the software architecture [18], and the integrator-internal architecture [19]. The software architecture specifies a dual messaging topology — NATS for the real-time path and Kafka for

the durable audit trail — that the thesis collapses into SQLite and FastAPI for single-machine evaluation, as discussed in Section 3.1 of Chapter Chapter 3.

The financial analysis [5] projects that monthly compute costs for research-active staff fall from approximately EUR 20/month in Phase 1 to approximately EUR 11/month in Phase 2, with frontier cloud pricing at approximately EUR 2.76 per million input tokens and EUR 13.80 per million output tokens, and on-premises amortised marginal cost at EUR 0.05 per million tokens. These numbers ground the cost model in Section 3.5 of Chapter Chapter 3. The use-case document [20] narrates a week of an institutional researcher and is the source of the Vignette 3.7 (Period-end project reporting) workflow that anchors the C1 benchmark: eight specialised agents gather reporting data across eight university systems with an estimated 80% cloud-payload reduction and a per-report cost of approximately EUR 2.15 in Phase 2.

Saleh *et al.* [61] introduce *MemIndex*, the FCG group's baseline architecture for distributed agent memory, which organises memory into event-driven layers with publish-subscribe semantics. The three-layer split (access, compression, storage) used in Chapter Chapter 3 is a direct extension of MemIndex with compression promoted to a first-class layer. The companion ACM Computing Surveys paper [62] surveys message brokers for generative AI, evaluating traditional and modern brokers against the GenAI workload. The survey's reference GenAI-agent architecture decomposes an agent into four components — environment interaction, perception, decision-making (the "brain": memory, sub-goal decomposition, chain-of-thought, self-critic), and actuation — with message brokers as the integration fabric. The planner-worker-critic loop of this thesis instantiates the perception–brain–actuation path as three concrete actors communicating through the memory bus.

The industry partner of this thesis, TalentAdore, supplies the production-pricing constants for the cost model in C3 (GPT-4o-mini and Claude Haiku 4.5) and is the source of the EUR-per-workflow framing for the cost arm. Their interest in this thesis is the cost-vs-coordination trade-off curve.

2.6. Synthesis and Positioning

The literature offers strong single-agent compression techniques (LLMLingua-2, ICAE, gist tokens, AutoCompressor), a maturing multi-agent runtime ecosystem (AutoGen, MetaGPT, CAMEL, Reflexion), a maturing RAG-plus-compression literature (xRAG, LongLLMLingua, dynamic compression for RAG), and an emerging privacy-aware retrieval literature (Privacy-Aware RAG, SecureRAG, SecurityLingua). It offers indirect evidence from industry that context management materially affects multi-agent performance (Anthropic's research-system writeup, Anthropic's context-engineering primitives).

What the literature does not offer is a systematic empirical characterisation of how compression degrades multi-agent coordination — whether the coordination curve is smooth like the single-agent QA curve, as Rae *et al.* [16] report for single-context compressed memory, or whether it has a cliff that single-agent metrics cannot anticipate. The thesis is positioned squarely on that gap, with the four contributions C1–C4 working together to make the question empirically tractable on a single workstation.

3. SYSTEM DESIGN AND IMPLEMENTATION

This chapter presents the system design and the implementation of the reference memory bus, the compressor framework, the C1 benchmark, the RAG pipeline catalogue, and the agent runtime. The intent is to give a reader enough detail to reproduce the system from the open-source repository without needing to refer back to the code, and to make the design choices explicit so that the experimental results in Chapter Chapter 4 can be interpreted against the constraints those choices imposed. The chapter closes with a section on the inference-backend stack, the single-machine compute envelope, and the reproducibility package that accompanies the manuscript.

3.1. System Architecture

3.1.1. Three-Layer Memory Bus

The reference implementation organises the memory bus into three layers that match the access pattern of a multi-agent workflow:

1. An **access layer**, exposing read, write, subscribe, and audit endpoints via FastAPI, and enforcing per-request access-control checks through a policy middleware.
2. A **compression layer**, providing a pluggable `Compressor` protocol behind which four concrete variants live: the identity (no-compression) control condition, the LLMLingua-2 hard-prompt baseline, the instruction-aware heuristic filter, the ICAE-style soft-prompt compressor, and the C4 tag-preserving variant.
3. A **storage layer**, providing three protocols — `AuditLog`, `Scratchpad`, and `VectorStore` — implemented for the thesis evaluation by SQLite in WAL mode, an in-memory time-to-live dictionary, and FAISS-CPU with an HNSW index.

The three layers are summarised in Figure 1. Every component dependency in the source tree is one-directional: the compression layer does not import from the access layer, and the storage layer does not import from either. This layering allows the storage backends to be replaced without touching the compression or access layers.

The three-layer structure is a direct extension of MemIndex [61], the FCG group's baseline architecture for distributed agent memory. The two material differences are that compression is promoted to a first-class layer rather than treated as a downstream optimisation, and that per-slot tags are part of the data model rather than out-of-band metadata.

3.1.2. Data Flow for a Write

A single write request flows through the bus in the following order. The client sends `POST /v1/write` with a JSON body containing the fragment text, its tag vector, and an optional task hint. The policy middleware parses the requester principal from the `X-M6-Principal` header and writes it to `request.state.principal` as a Starlette `State()` attribute. The service receives the request, checks that the principal's access-control mask is a superset of the

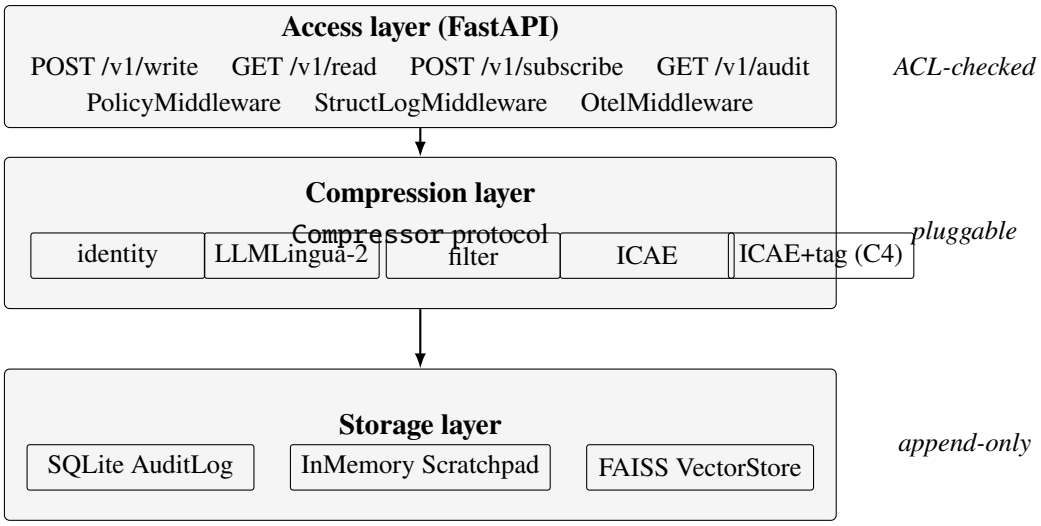

Figure 1: Three-layer architecture of the memory-bus reference implementation. Each layer depends only on the layer below it, and each storage component is hidden behind a `Protocol` so that the storage backends can be replaced without touching the access or compression layers.

fragment tag's mask and that the principal's classification is at least the fragment's classification, calls the active compressor's `compress(fragment, target_ratio)` method to produce a `CompressedSlot`, places that slot in the in-memory scratchpad keyed on the slot identifier, computes an embedding for the slot if the compressor exposes one and adds it to the FAISS index, and finally appends a row to the audit log whose `event_type` is `WRITE`, `result` is `OK`, `prev_hash` is the previous row's chain hash, and `chain_hash` is SHA-256(`prev_hash` ‖ `payload_hash`). The response carries the slot identifier, the chain hash hex-encoded, and the audit row identifier.

If the policy check fails, the bus appends a `DENY` row with the synthesised slot identifier `deny-<uuid8>`, with a 60-second per-*(subject, fragment-id)* deduplication so that an adversarial client hammering denied writes cannot inflate the audit log.

3.1.3. Data Flow for a Read

A read request flows symmetrically. The client sends `GET /v1/read/{slot_id}`, the policy middleware attaches the principal, the service looks up the slot in the scratchpad, checks the principal–tag relationship, and either returns the slot together with the audit row identifier or raises a policy-denied or slot-not-found error. The slot-not-found path is itself audited with a 60-second per-*(subject, slot-id)* deduplication: the first read miss for a given subject and slot is appended to the audit log as a `READ/ERROR` row, and subsequent misses within the window short-circuit silently so that an adversarial reader cannot inflate the audit log either.

3.1.4. Boundary with the FCG Platform

The FCG software architecture [18] specifies a dual messaging topology: NATS for the real-time path and Kafka for the durable path. The thesis collapses these into SQLite, an in-memory dictionary, and an HTTP API for two reasons. First, the compute envelope is one machine:

running NATS and Kafka on the same M4 Pro adds operational complexity without adding empirical signal. Second, the FCG architecture document budgets the total synchronous epoch path at 150–200 ms end-to-end [18], which SQLite and FastAPI sit comfortably under on a single host. The contract that the FCG team can adopt is the FastAPI surface; the wire protocol is a separate concern.

3.2. Data Model

3.2.1. Tags, Classifications, and Access Control

Every fragment and every compressed slot carries a tag vector that records its provenance and access-control state. The tag vector has four fields: a `uint64` access-control mask, a 5-tier classification level (PUBLIC < INTERNAL < CONFIDENTIAL < RESTRICTED < SECRET) drawn from the NIST AI Risk Management Framework [25], a tuple of source identifiers recording provenance, and a tuple of parent-slot identifiers recording which previously-compressed slots this slot inherits from.

Tag aggregation respects the lattice: when slots are merged or when a new slot inherits from several parents, its access-control mask is the bitwise OR of the parents' masks and its classification is the maximum. The lattice rules are encoded in the `TagVector.union(...)` classmethod and unit-tested. The policy predicate is symmetric for reads and writes: a requester satisfies a tag if and only if every required bit is in the requester's mask and the requester's classification is at least the tag's classification.

3.2.2. Compressed Slot Payload Variants

The compressed slot carries a tagged-union payload because the three compressor families produce three different representations: a literal text summary for the LLMLingua-2 and instruction-aware filter families, a soft embedding tensor of shape (M, d_{model}) for the ICAE family, and an integer token-identifier tuple for the AutoCompressor-style variant. Storage and access layers never branch on the payload kind; the embedding interface exposed by the compressor is the only place where the kind matters.

3.2.3. Audit Log

The audit log is an append-only SQLite table with the columns shown in Table 1. An `INSTEAD OF UPDATE` and an `INSTEAD OF DELETE` trigger refuse any mutation of an existing row. The chain-hash trigger is verified by a dedicated unit test that drops the trigger to simulate a file-level tamper, edits a single `payload_hash`, and confirms that the `verify()` chain-walker reports failure.

Table 1: Schema of the SQLite `audit_log` table. Triggers prevent UPDATE and DELETE on this table at the database level.

Column	Type	Notes
`rowid`	INTEGER PK	monotonic, auto-increment
`slot_id`	TEXT NOT NULL	matches the slot identifier
`event_type`	TEXT CHECK	one of WRITE / READ / SUBSCRIBE / DENY / COMPRESS / EVICT
`requester_acl`	INTEGER	64-bit ACL mask, stored signed
`requester_class`	INTEGER	classification level 0–4
`result`	TEXT CHECK	OK / DENIED / ERROR
`prev_hash`	BLOB(32)	SHA-256 of the previous row's chain_hash
`payload_hash`	BLOB(32)	SHA-256 of the event's payload bytes
`chain_hash`	BLOB(32)	SHA-256(prev_hash ‖ payload_hash)
`created_at`	TEXT NOT NULL	ISO-8601 UTC timestamp

3.2.4. Cost Ledger

A second SQLite table records every paid model call: provider, model, input tokens, output tokens, EUR cost, and wallclock. The cost ledger shares the audit-log database for transactional consistency but is queried separately. The pricing table is a single Python module (`m6.pipelines.cost_model`) whose constants are grounded in the financial analysis [5]: the frontier-cloud reference price is USD 3 per million input tokens and USD 15 per million output tokens, which corresponds to approximately EUR 2.76 and EUR 13.80 respectively; the on-premises amortised marginal cost is EUR 0.05 per million tokens. The GPT-4o-mini and Claude Haiku 4.5 prices are drawn from TalentAdore production accounts as of the writing of this thesis.

3.3. Compressor Framework

3.3.1. The `Compressor` Protocol

Every compressor variant conforms to a Python `Protocol` with four methods — `compress(fragment, target_ratio)` returning a `CompressedSlot`; `decompress(slot)` returning a best-effort text reconstruction; `embed(slot)` returning an optional embedding for vector-store indexing; and `model_card()` returning a structured card containing the compressor identifier, family, base model, training-loss formula, default target ratio, and provenance hashes. The thesis ships five concrete implementations: the identity (no-compression) control condition, LLMLingua-2, the instruction-aware filter, the ICAE-style soft-prompt compressor, and the tag-preserving variant that subclasses ICAE.

The identity compressor is the no-compression control condition required by every experiment as a mitigation against runtime-bug confounds. The experimental protocol in Chapter Chapter 4 flags an experiment as `invalid` whenever the variance of the control condition is comparable to or exceeds the variance of the treatment of interest.

3.3.2. LLMLingua-2 Wrapper

The LLMLingua-2 wrapper is a thin adapter around the `llmlingua` package [10]. Construction takes a target compression ratio; the package is asked to retain $1/r$ of the tokens. The force-tokens list (sentence boundary marks) is validated against the wrapped tokenizer's vocabulary at initialisation and unknown tokens are dropped with an explicit log line; this avoids silent compression-quality degradation when the wrapped model's tokenizer does not include a punctuation mark on the default list.

Calibration target: NarrativeQA F1 within 2 pp of the reported numbers [30] and HotpotQA F1 within 3 pp of the reported numbers [31]; failure to meet these targets triggers a stop-the-line review.

3.3.3. Instruction-Aware Filter

The instruction-aware filter is a pure-Python heuristic baseline that serves as the floor against which the trained ICAE compressor must demonstrate a benefit. It operates in two stages: first a sentence-level filter that ranks sentences by their relevance to the task hint using either a cross-encoder reranker (`BAAI/bge-reranker-base`) when available or a lexical-overlap fallback when not; second a token-level trim that drops stop-words and short tokens until the target compression ratio is reached. The filter is deterministic given the task hint, the fragment, and the target ratio. It carries no training cost and is therefore the most economical baseline; the trained ICAE compressor must beat it by at least 3 pp F1 to justify its training cost.

3.3.4. ICAE-Style Soft-Prompt Compressor

The trainable arm of the thesis follows the ICAE construction [13] for the encoder–decoder architecture and adds a contrastive term inspired by the segment-level language-modelling objective of AutoCompressor [12]:

$$L = L_{\text{recon}} + \lambda \cdot L_{\text{NCE}} \tag{1}$$

The architecture (Section 3.3.6) uses Llama-3.1-8B-Instruct [63] as both encoder and decoder. The encoder is LoRA-adapted [64] with rank 16, alpha 32, and dropout 0,05 on the seven canonical target modules (`q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, `down_proj`). The decoder is frozen. The memory consists of $M = 128$ learnable special tokens appended to the encoder input; their last-layer hidden states are the compressed representation. The reconstruction loss L_{recon} is the standard token-level cross-entropy of the source fragment given the compressed memory slots, computed on the frozen decoder.

The contrastive loss L_{NCE} is an InfoNCE loss [33] over positive and negative source-fragment pairs. Positive pairs are drawn from fragments of the same source document. Negative pairs are other fragments in the same batch (in-batch negatives), and the temperature is 0,07 following SimCLR [34]. The weight λ is searched in $\{0{,}1, 0{,}3, 1{,}0\}$ on a validation split.

The InfoNCE primitive in MLX is not available as of version 0,21, so the trainer probes for it at startup and falls back to a PyTorch-MPS implementation if the probe fails; this is the documented mitigation for the second risk-register row of the implementation plan. The

`infonce_loss` function is unit-tested against a numpy reference implementation to confirm numerical equivalence.

3.3.5. Tag-Preserving Variant (C4)

The C4 variant is a subclass of the ICAE compressor that adds a per-slot tag-prediction head. The tag head is a two-output MLP that emits, from each pooled memory-slot embedding, a 64-dimensional logit vector for the access-control mask and a 5-dimensional logit vector for the classification level. The training loss extends Equation 1 with a tag-prediction term:

$$L = L_{\text{recon}} + \lambda_{\text{NCE}} \cdot L_{\text{NCE}} + \lambda_{\text{tag}} \cdot L_{\text{tag}}, \tag{2}$$

where L_{tag} is binary cross-entropy on the ACL output and cross-entropy on the classification output. The weight λ_{tag} is searched in $\{0{,}1, 0{,}3, 1{,}0, 3{,}0\}$. Tag recovery at inference time is performed slot by slot: ACL bits are recovered via a 0.5 sigmoid threshold and aggregated across slots by bitwise OR; the classification is recovered as the argmax of the softmax and aggregated by taking the maximum across slots. The aggregation is lattice-respecting by construction.

The H4 evaluation defines a fragment's tag as *preserved* when either the recovered ACL is a superset of the source ACL or $|\text{recovered} \cap \text{source}|/|\text{source}| \geq 0{,}9$, and the recovered classification is at least the source classification.

3.3.6. Training Configuration

Optimiser AdamW with learning rate 3×10^{-4}, weight decay 0, $\beta_1 = 0{,}9$, $\beta_2 = 0{,}95$, cosine schedule with 200-step linear warmup. Batch size 4, gradient accumulation 8, sequence length 512 (fragment) plus 128 (memory). Wallclock budget at most three days on the M4 Pro 48 GB for a 7B-parameter encoder at rank 16.

3.4. The C1 Benchmark

3.4.1. Workload Families

The C1 benchmark generator produces three workload families:

Family (a) — cross-document fact aggregation. Eight source documents per instance, each representing one institutional system in the Vignette 3.7 inventory (Publication DB, Contract DB, Moodle, Patio, Peppi, CRM, Tatu SAP, SAP Travel [20]); the planner is asked to aggregate hours and budget across the eight sources. The aggregation is deterministic by construction so that the ground truth is reproducible from the seed; the critic checks the planner's answer against the aggregate.

Family (b) — constraint-satisfaction planning. N sub-tasks must be assigned to K capacity-bounded workers under load-cum-capacity constraints; the workload generator produces a feasible solution via a greedy capacity-respecting algorithm that bumps capacities only when strictly necessary and retries the same sub-task, guaranteeing that every workload has at least one feasible assignment.

Family (c) — multi-step retrieval. A linear chain of fragments where each fragment points to the next and the leaf carries the answer; chain lengths range from 2 to 4 and each fragment is padded with 4 to 8 distractor sentences so that the retrieval step is non- trivial.

Each family produces approximately 50 instances per generation seed for a total of 150 instances per release. The release is reproducible from a single command and the configuration hash is recorded in the manifest for forensic comparability.

3.4.2. Synthetic Tag Distributions

The thesis uses synthetic tags because real ACL data was not available on the master's-scale schedule. The generator supports three tag distributions: *uniform*, in which each ACL bit is set independently with probability 0,5; *skewed*, in which a handful of bits dominate via an exponential family chosen to mimic typical production ACL skews; and *hierarchical*, in which higher classification levels imply strict supersets of the bits at lower classification levels. The H4 evaluation varies the distribution to probe the governance-stringency dimension explicitly.

3.4.3. Coordination-Quality Metrics

The coordination scorer is a pure function of the workload and the trace; it does not call the model. It returns four numbers: *final task success* (boolean, family-specific predicate), *sub-task assignment accuracy* (the fraction of sub-tasks assigned to the worker the workload's ground truth expects), *rounds to completion* (the number of planner-worker-critic cycles before the planner declares `DONE`), and *critic-flagged error rate* (the fraction of rounds in which the critic returned `REVISE`).

The success predicate for family (a) is an exact match on the aggregate, for family (b) is a constraint-satisfaction check against the workload's advertised capacities, and for family (c) is an exact match on the leaf answer. The constraint-satisfaction predicate fails closed when the workload's metadata is missing the capacities field, which would otherwise be an opportunity for the planner to pass trivially under a generator bug.

3.5. RAG Pipeline Catalogue

The C3 catalogue consists of three pipelines, each implemented on top of FAISS-CPU and LlamaIndex [21, 22].

3.5.1. P1: Compress, Then Retrieve

The corpus is compressed up-front; the FAISS index stores the compressed text or, for soft-prompt compressors, the mean-pooled memory-slot embeddings. Retrieval and synthesis operate on the compressed representation. P1 trades quality for storage efficiency: information dropped at compression time is gone, but the index is small. The pre-processing baseline against which P1 is informally compared is Anthropic's contextual-retrieval [52], which prepends LLM-generated chunk-level explanatory context before embedding.

3.5.2. P2: Retrieve, Then Compress

The corpus is indexed in full; the compressor runs as a post-retrieval node-processor over the top-k retrieved fragments before they are synthesised. This is the classical LongLLMLingua configuration [9]. P2 trades storage cost for retrieval recall: the index is the full corpus, but the synthesis context can be compressed aggressively because the retriever has already identified the relevant chunks.

3.5.3. P3: Joint, Conditional on Relevance

The corpus is indexed in full. At query time, each retrieved fragment is routed into one of three branches based on its retrieval relevance score s: fragments with $s \geq \theta_{\text{high}}$ are passed verbatim, fragments with $\theta_{\text{low}} \leq s < \theta_{\text{high}}$ are compressed, and fragments with $s < \theta_{\text{low}}$ are discarded. The thresholds θ_{high} and θ_{low} are swept over a small grid; the default values 0,75 and 0,45 on `BAAI/bge-large-en-v1.5`-cosine similarities are the analytical starting points. P3 follows the structural design of [23].

3.5.4. Cost Model

The pipeline cost model is a single Python module whose pricing constants are grounded in the financial-analysis document [5]: the frontier-cloud reference price is USD 3 per million input tokens and USD 15 per million output tokens, or approximately EUR 2.76 and EUR 13.80. The on-premises amortised marginal cost is EUR 0.05 per million tokens. The GPT-4o-mini and Claude Haiku 4.5 prices reflect TalentAdore production accounts as of the writing of this thesis. The headline cost metric in the C3 evaluation is EUR per workflow, with a corollary headline of EUR per period-end report against the Vignette 3.7 target of approximately EUR 2.15 [20].

3.6. Agent Runtime

The agent runtime uses AutoGen v0.4 pinned in the `requirements.txt` [1]. The planner-worker- critic loop is built as a `RoundRobinGroupChat` over a planner, a configurable number of workers, and a critic that emits a structured `DONE` or `REVISE` verdict. The scratchpad is the memory bus: every agent write goes through `POST /v1/write` and every agent read goes through `GET /v1/read/{slot_id}`, so the compressor is on the critical path and the experimental knob is what the experiment manipulates rather than a hidden detail of the runtime.

A second backend, `determ`, is a deterministic family-specific solver that reads the (optionally compressed) fragments and produces a trace directly. Its purpose is not to replace AutoGen as the production runtime; it is the *upper bound* on coordination success that isolates compression-quality variance from runtime variance. If the AutoGen runner fails to match the deterministic runner on the no-compression control condition, the experiment is flagged invalid in the manifest.

The orchestrator caches the constructed compressor by $(\text{name}, \text{ratio})$ so that a trained ICAE compressor is instantiated once per cell rather than once per workload–seed pair; without this cache the model would re-load on every inner-loop iteration of H2 (approximately twenty-two thousand iterations per run).

3.7. Inference Backends

The unified `InferenceBackend` protocol exposes a single `complete(prompt, max_tokens, temperature, stop)` method, together with an optional `embed(text)` method and a cost predicate that consults the pricing table. Four concrete backends ship: MLX-LM for 7B and 13B inference on Apple Silicon [26]; Ollama wrapping a localhost HTTP endpoint for developer-iteration speed [28]; OpenAI for GPT-4o-mini as the cost-arm comparator; and Anthropic for Claude Haiku 4.5 as the cost-arm comparator and Claude Sonnet 4.6 as the cross-check on the 10% sample. A fifth backend, `echo`, is a deterministic mock used in unit tests.

The OpenAI backend dispatches between `max_tokens` (GPT-4o-mini and other non-o-series models) and `max_completion_tokens` (o-series) on the model name; passing the wrong parameter is rejected by the API in some SDK versions.

3.8. Compute Envelope and Reproducibility

All training and evaluation is run on a single Apple M4 Pro 48 GB workstation. The breakdown of expected wallclocks is given in Table 2.

Table 2: Expected wallclock for each experimental arm on the M4 Pro 48 GB workstation. Numbers in parentheses indicate stretch goals that will be dropped if mid-thesis review at Month 5 reveals schedule slippage. From the implementation plan.

Experiment	Approach	Wallclock
NarrativeQA/HotpotQA 7B baseline reproduction	MLX/PyTorch-MPS	1–2 days
C1 baseline at 7B: 3 compressors $\times$ 5 ratios $\times$ 5 seeds $\times$ 150 workflows	MLX, AutoGen	5–10 days
ICAE-style trainer, 7B LoRA rank 16	MLX	$\leq$ 3 days
H2 secondary arm: 13B fp16 on 30-workload subsample	MLX-LM	3–5 days
Tag-preserving ICAE training (C4)	MLX, rank 16	$\leq$ 3 days
RAG pipeline benchmark (H3)	MLX inference + FAISS	3 days
Tag preservation sweep (H4)	MLX inference	1 day
Adversarial-robustness sweep (S2, stretch)	7B	1 week

The reproducibility package is the complete repository, a `docker-compose.yml` that brings the memory-bus reference service up, a GitHub release tag for the manuscript and code, model and data cards under `docs/`, and a `README.md` that exposes one-command reproduction for every chapter headline figure. The filesystem layout under `results/` is fixed: one CSV per hypothesis run, plus a `manifest.yaml` that records the resolved configuration, the configuration hash, the git revision, the Python version, the platform string, and the start time.

4. EXPERIMENT DESIGN AND PROTOCOL

This chapter specifies the experimental protocol for each of the four hypotheses, the metric suite, the statistical-analysis plan, and the schedule of arms. Each hypothesis section gives the falsifiable claim verbatim, the inputs the runner takes, the procedure, the decision rule that maps the run's output to a verdict, and the wallclock estimate. Result sections are explicit placeholders marked with the LaTeX comment `% RESULTS`: they will be populated by the experimental runs and are intentionally empty in this submission. The empirical findings will be inserted into those slots verbatim from the `results/h{N}/<run_id>/` directory and the corresponding `verdicts.json`.

4.1. Metric Suite

The metric suite covers seven families. The first three are quality-oriented and standard; the next two are coordination- and compression-specific; the last two are operational.

Quality. F1 (token-overlap), exact match (EM), ROUGE-L, and BERTScore, all computed by pure-Python implementations against ground-truth answers [65–67]. The F1 implementation matches the SQuAD reference, which is the version reproduced by the LLMLingua-2 paper and the basis for the calibration target.

Coordination. Final task success, sub-task assignment accuracy, rounds to completion, and critic-flagged error rate. All four are pure functions of the workload and the trace, as documented in Section 3.4 of Chapter Chapter 3.

Faithfulness and hallucination. RAGAS [68] and DeepEval [69] faithfulness scores, plus manual annotation on a 100-example random sample for spot-check calibration.

Compression. Input-to-output token ratio measured on the wrapped tokenizer of the active compressor; total tokens per workflow; per-stage compression ratio when more than one stage applies (as in P2 and P3).

Cost. EUR per query and EUR per workflow, computed from the cost ledger using the pricing constants of Section 3.5 of Chapter Chapter 3.

Latency. p50 and p95 milliseconds per query, computed separately for the compression step and end-to-end.

Governance. Tag-preservation rate (per the H4 definition).

LLM-as-judge. A subset of metrics is scored by a local Llama-3.1-8B-Instruct (via Ollama) as the default judge model. The opt-in cost-comparison arm switches the judge to GPT-4o-mini with a 10% sample cross-checked by Claude Sonnet 4.6; Cohen's κ below 0,85 between the two judges triggers a recalibration. The judge sees a position-randomised pair of answers and

never sees which compressor produced which answer. Every headline figure in this thesis is reproducible without any external API key; the paid judge is only used to populate the Chapter 6 cost-comparison panel.

4.2. Statistical Analysis Plan

The pre-registered analysis plan is the one specified in the implementation plan and the technical reference.

Seeds. Five seeds per condition.

Confidence intervals. Empirical 95% bootstrap CI with 10 000 resamples, computed via `bootstrap_mean_ci` in `m6.evaluation.statistics`.

Paired comparisons. For compressor-versus-compressor comparisons on matched instances, the paired bootstrap. The implementation uses the recentred-null percentile bootstrap p-value of Efron and Tibshirani [70] (Chapter 16): a two-sided test of H_0: mean difference = 0 is computed by re-centring the bootstrap distribution around its own mean and reporting the fraction of recentred bootstrap means whose absolute value meets or exceeds the observed absolute mean.

Cliff-detection test (H2). Mann–Whitney U for the comparison of success-below-τ versus success-above-τ. These two samples are independent because they are drawn across different (workload, seed, ratio) cells; a paired test would be incorrect [71, 72].

Multiple-comparison correction. Holm-Bonferroni within each hypothesis family [73]. The families are $\{H_1, H_2\}$, $\{H_3\}$, and $\{H_4\}$.

Effect sizes. Cliff's δ for ordinal comparisons [74] and Cohen's d for continuous ones, reported alongside p-values.

Validity. Every experiment includes a no-compression control condition. If the variance of the control on the metric of interest meets or exceeds the variance of the treatment, the experiment is flagged `invalid` in the run's manifest and the verdict is suppressed. This is the documented mitigation for the AutoGen-runtime-bug risk.

4.3. Hypothesis H1 --- QA Is a Poor Predictor of Coordination

Falsifiable claim. Spearman ρ between the single-agent QA-accuracy delta and the multi-agent coordination-success delta across matched compression ratios is below 0,6 on at least two of the three compressors at 7B.

Inputs. The C1 benchmark at version v0.1; the three baseline compressors; the MLX-LM backend at the 7B size.

Procedure. For each (compressor c, ratio r, workload w, seed s) cell: (i) build the single-agent QA derivative of the workload (same sources, planner-only question, no workers or critic) and record F1; (ii) run the planner-worker-critic loop and record coordination success, sub-task accuracy, rounds, and critic-flagged rate; (iii) compute $\Delta_{\text{qa}} = qa(r) - qa(r{=}1)$ and $\Delta_{\text{coord}} = \text{coord}(r) - \text{coord}(r{=}1)$. For each compressor compute Spearman ρ between Δ_{qa} and Δ_{coord}, with bootstrap CI.

Decision. Supported if $\rho < 0{,}6$ on at least two of the three compressors at 7B, with the bootstrap CI excluding 0,6 from above on the same two compressors. Falsified if $\rho \geq 0{,}6$ on at least two compressors. Both verdicts are thesis-worthy.

Wallclock. Approximately two days at 7B fp16.

4.4. Hypothesis H2 --- Coordination Cliff Exists

Falsifiable claim. For each (compressor, workload) cell at 7B, there is a compression ratio τ^* such that coordination success at ratios $r > \tau^*$ falls by at least 30% relative to ratios $r < \tau^*$. τ^* varies by workload but not by compressor family, within ±20%.

Inputs. The C1 benchmark, the three baseline compressors, the MLX-LM backend at 7B and 13B (the latter on a 30-workload sub-sample for sanity).

Procedure. Sweep $r \in \{1, 2, 3, 4, 5, 6, 8, 10, 12, 16\}$ on each cell with five seeds. Fit a piecewise-linear function $f(r; a, b, \tau)$ to the (ratio, success) curve via differential evolution under the constraint that the right-piece slope is at most the left-piece slope (the cliff drops). Run a Mann–Whitney U test on success-below-τ versus success-above-τ with the "greater" alternative, then apply Holm correction within the $\{H_1, H_2\}$ family. A secondary arm repeats the sweep at 13B on a 30-workload sub-sample to check whether τ^* shifts with model size.

Decision. A cliff exists at a cell if the relative drop is at least 0,30 and the Holm-adjusted Mann–Whitney p-value is less than 0,05. The τ-spread-by-workload check passes for a family when the (max-min)/mean of τ^* across compressors is at most 0,20.

Wallclock. Approximately 5–10 days at 7B plus 3 days at 13B sub-sample.

4.5. Hypothesis H3 --- RAG Pipeline Placement Matters

Falsifiable claim. P1 dominates P2 on storage-bounded settings; P2 dominates P1 on accuracy-bounded settings; P3 closes the gap on a combined accuracy-times-EUR score. Effect size of at least 5 pp F1.

Inputs. The C1 family-(a) workloads plus NarrativeQA and HotpotQA for external comparability; the three pipelines P1, P2, and P3; the three compressors.

Procedure. Two budget regimes are defined: *storage-bounded*, with the FAISS index size capped at 100 MB so that both P1 and P2 are tuned to fit; and *accuracy-bounded*, with retrieval recall@10 capped at 0,85 so that both P1 and P2 are tuned to meet that ceiling. For each $(\text{pipeline}, \text{regime}, \text{ratio} \in \{2, 4, 8\}, \text{seed})$ cell run the full workload and record F1, EUR per query, latency p50 and p95, FAISS index megabytes, and recall@10. P3 is swept over $(\theta_{\text{high}}, \theta_{\text{low}})$ on the small grid documented in Section 3.5 of Chapter Chapter 3.

Decision. P1-versus-P2 effect-size at least 5 pp F1 with paired-bootstrap CI excluding 0 in both regimes (with the sign flipping per the H3 claim); P3 wins on a combined F_1/EUR ranking with Holm-adjusted p less than 0,05.

Wallclock. Approximately three days.

4.6. Hypothesis H4 --- Tag Preservation

Falsifiable claim. At 4× compression the tag head preserves at least 85% of fragment tags, with at most 5 pp accuracy degradation relative to the unmodified baseline. The CI lower bound on the accuracy delta is used as the strict test (not the point estimate).

Inputs. The ICAE baseline checkpoint and the C4 tag-preserving checkpoint; the C1 benchmark across the three tag distributions (uniform, skewed, hierarchical).

Procedure. For each $r \in \{2, 4, 8\}$ and each workload, compress every fragment with the C4 variant and recover its tag via the tag head. The preservation predicate combines the ACL test (either the recovered ACL is a superset of the true ACL or the intersection-over-true ratio is at least 0,9) and the classification test (the recovered classification is at least the true classification). For the accuracy delta, run the planner-worker-critic loop with the baseline and the C4 compressor on the same workloads with the same seeds and compute the paired bootstrap of the coordination-success difference.

Decision. Supported if the mean preservation rate at 4× is at least 0,85 and the paired-bootstrap CI lower bound on the accuracy delta is at least -5 pp.

Wallclock. Approximately one day.

4.7. Headline Figures

The three chapter-headline figures the manuscript will produce are listed in Table 3; each figure is a single command (`make reproduce-ch<N>`) that lands in `results/h{X}/<run_id>/plots/`.

Table 3: Chapter-headline figures and their reproduction targets.

Chapter	Figure	Reproduction
5	Coordination cliff τ^* at 7B (+ 13B sanity)	`make reproduce-ch5` (H1, H2)
6	RAG pipeline placement and cost	`make reproduce-ch6` (H3)
7	Tag preservation	`make reproduce-ch7` (H4)

4.8. Threats to Validity

Construct validity. The C1 benchmark is synthetic. Real-trace transfer validation is listed as future work in Chapter Chapter 5.

Internal validity. The no-compression control condition guards against AutoGen-runtime bugs confounding the compression-quality signal. The deterministic backend provides a second upper bound that any AutoGen run must match.

External validity. The 13B sanity arm in H2 provides a secondary data point on model-size sensitivity. The primary evaluation is at 7B, which is the largest model that fits comfortably in fp16 on the M4 Pro 48 GB workstation.

Statistical validity. The recentred-null percentile-bootstrap p-value is more rigorous than the tail-doubling shortcut and is consistent with the recommendations of Efron and Tibshirani [70]. The Mann–Whitney U test for the cliff-detection check matches the independent-samples structure of the data. Holm correction is applied per hypothesis family to control the family-wise error rate.

Reproducibility. Every experiment writes a `manifest.yaml` that pins the configuration hash, the git revision, the Python version, and the platform string. Every CSV is reproducible from a single command. The pricing constants live in one Python module and are updated in one place.

5. DISCUSSION AND SUMMARY

This chapter draws together the thesis as it stands at submission. The empirical narrative that connects the four hypotheses, the quantitative findings, and the comparison against the falsification targets is the content of the discussion proper, and will be inserted here once the experimental runs have finished and the verdicts are in. The structure of the discussion is fixed: the chapter revisits the four contributions in order, summarises the empirical findings or reports the falsifications, then moves to the limitations, the deliverables, and the closing remarks.

5.1. Summary of Empirical Findings

H1.

H2.

H3.

H4.

5.2. Stretch Goals

The implementation plan reserves Month 8 for at most two stretch goals selected at the end of Month 7 from a pre-agreed list:

S1 — Calibration study. Fit the piecewise-linear cliff on ≤ 100 workload instances and use it to predict τ^* on held-out workflows. Target mean absolute error of at most 1,5 ratio units. Adds a practical-deployability section to Chapter Chapter 4.

S2 — Adversarial robustness. A SecurityLingua-style red-team sweep against the tag-preserving compressor [59]. Reports tag-preservation under adversarial prompts. Hardens the C4 governance claims.

S3 — Haseeb baseline. Implement the multi-agent context- engineering approach of Haseeb [45] as a baseline comparator on C1 against the C2 compressor suite. Strengthens Chapter Chapter 2.

S4 — TalentAdore trace replication. With Asim Nadeem and Oskari Valkama, replicate the C1 evaluation on an NDA-cleared TalentAdore production trace. Strengthens the industry-relevance arm.

5.3. Limitations

Synthetic benchmark. The C1 benchmark is synthetic. Real-trace transfer validation is listed as future work.

Single workstation. All training and evaluation fit a single Apple M4 Pro 48 GB workstation. The primary evaluation is at 7B with a 13B sanity check.

Tag-preserving compressor as a research prototype. The tag-preserving compressor is a research prototype with measured properties, not a compliance tool. It is not certified against any particular ACL or governance framework. The use of the NIST AI RMF [25] lattice in the tag vector borrows only the vocabulary.

Statistical power. Five seeds per cell gives modest power for very small effect sizes. The Holm-Bonferroni correction is conservative on hypothesis families with many members. The thesis chooses both anyway: the alternative (under-correcting and over-reporting) would be worse for a manuscript whose results must be defensible.

Audit log. The tamper-evident hash chain in the audit log detects in-process tampering but does not defend against file-level attacks on the SQLite database.

Agent runtime version pinning. AutoGen v0.4 is pinned. A minor-version bump during the thesis window could change behaviour; the pinning is the documented mitigation and the no-compression control condition is the second.

5.4. Future Work

The thesis leaves several natural extensions open:

Real-trace transfer. The synthetic C1 benchmark could be validated against real deployment traces from institutional systems (M0/M1/M2 connectors at the University of Oulu) to confirm that the coordination-cliff findings transfer to production workloads.

Model-size scaling. The primary evaluation at 7B and the 13B sanity arm leave open the question of how τ^* shifts at larger model sizes (34B, 70B). Quantised inference via llama.cpp would make this feasible on commodity hardware.

Training-distribution ablation. The thesis trains the ICAE compressor on a mixed corpus. A controlled comparison between a dialogue-trace-trained variant and a QA-trained variant at matched accuracy would isolate the effect of training distribution on coordination quality.

Distributed deployment. The storage protocols (`AuditLog`, `Scratchpad`, `VectorStore`) are designed as pluggable interfaces. Replacing SQLite with Postgres, the in-memory scratchpad with Redis Cluster, and FAISS with Qdrant would enable a multi-machine deployment without changing the compression or access layers.

Cross-tokenizer exchange. The compressor interface carries a `tokenizer_id` field to support future cross-tokenizer or cross-model-family compressed exchange, which is out of scope for this thesis.

5.5. Deliverables

The thesis is delivered together with: the open-source reference implementation hosted on GitHub at a release tag matching the manuscript version, the C1 benchmark v0.1 reproducible from a single command, the tag-preserving compressor weights released with model and data cards, a Docker compose deployment for the memory-bus service, and a one-command reproduction target for each chapter-headline figure (`make reproduce-ch{5..7}`). The reproducibility package is the final deliverable; the manuscript is the second-final.

5.6. Closing

6. REFERENCES

[1] Q. Wu et al., *AutoGen: Enabling next-gen LLM applications via multi-agent conversation*, arXiv preprint arXiv:2308.08155, 2023.

[2] S. Hong et al., "MetaGPT: Meta programming for a multi-agent collaborative framework", in *Proc. International Conference on Learning Representations (ICLR)*, 2024.

[3] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning", in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[4] Anthropic. "How we built our multi-agent research system", Anthropic Engineering Blog, Accessed: May 12, 2026. [Online]. Available: `https://www.anthropic.com/engineering/multi-agent-research-system`.

[5] Future Computing Group, "Financial analysis: University AI service economy", CSE Research Unit, University of Oulu, Tech. Rep., 2026, Internal document, March 2026. Five-year projection across the University of Oulu.

[6] P. Rajasekaran, K. Dixon, J. Ryan, A. Hadfield, et al. "Effective context engineering for AI agents", Anthropic Applied AI Engineering Blog, Accessed: May 12, 2026. [Online]. Available: `https://www.anthropic.com/engineering/context-engineering-for-ai-agents`.

[7] Anthropic. "Managing context on the Claude developer platform: Context editing and the memory tool", Anthropic News, Accessed: May 12, 2026. [Online]. Available: `https://www.anthropic.com/news/context-editing-memory-tool`.

[8] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu, "LLMLingua: Compressing prompts for accelerated inference of large language models", in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[9] H. Jiang et al., "LongLLMLingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression", in *Proc. Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

[10] Z. Pan et al., "LLMLingua-2: Data distillation for efficient and faithful task-agnostic prompt compression", in *Findings of the Association for Computational Linguistics (ACL Findings)*, 2024.

[11] J. Mu, X. L. Li, and N. Goodman, "Learning to compress prompts with gist tokens", in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[12] A. Chevalier, A. Wettig, A. Ajith, and D. Chen, "Adapting language models to compress contexts", in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, AutoCompressor., 2023.

[13] T. Ge, J. Hu, L. Wang, X. Wang, S.-Q. Chen, and F. Wei, "In-context autoencoder for context compression in a large language model", in *Proc. International Conference on Learning Representations (ICLR)*, 2024.

[14] Wang et al., “In-context former: Lightning-fast compressing context for large language model”, in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2024.

[15] Z. Li, Y. Liu, Y. Su, and N. Collier, “Prompt compression for large language models: A survey”, in *Proc. Conf. North American Chapter of the Association for Computational Linguistics (NAACL)*, 2025.

[16] J. W. Rae, A. Potapenko, S. M. Jayakumar, C. Hillier, and T. P. Lillicrap, “Compressive transformers for long-range sequence modelling”, in *Proc. International Conference on Learning Representations (ICLR)*, 2020.

[17] Future Computing Group, “System architecture for real-time AI service markets”, CSE Research Unit, University of Oulu, Tech. Rep., 2026, Internal document, March 2026.

[18] Future Computing Group, “Neutral AI service exchange software architecture”, CSE Research Unit, University of Oulu, Tech. Rep., 2026, Internal document, March 2026.

[19] Future Computing Group, “Integrator system architecture”, CSE Research Unit, University of Oulu, Tech. Rep., 2026, Internal document, March 2026.

[20] Future Computing Group, “Use case: A week at the university in the AI service economy”, CSE Research Unit, University of Oulu, Tech. Rep., 2026, Internal document, March 2026. Vignette 3.7 (Period-end project reporting) is the focal scenario.

[21] J. Johnson, M. Douze, and H. Jégou. “Faiss: A library for efficient similarity search and clustering of dense vectors”, Facebook AI Research, Accessed: May 12, 2026. [Online]. Available: `https://github.com/facebookresearch/faiss`.

[22] J. Liu and LlamaIndex Team. “LlamaIndex: A data framework for LLM applications”, Accessed: May 12, 2026. [Online]. Available: `https://github.com/run-llama/llama_index`.

[23] Guo et al., *Dynamic context compression for retrieval-augmented generation*, arXiv preprint, 2025.

[24] Park et al., *Collaborative memory: Multi-user memory sharing in LLM agents with dynamic access control*, arXiv preprint arXiv:2505.18279, 2025.

[25] National Institute of Standards and Technology, “AI risk management framework (AI RMF 1.0)”, U.S. Department of Commerce, Tech. Rep., 2023.

[26] A. Hannun, J. Digani, A. Katharopoulos, and R. Collobert. “MLX: Efficient and flexible machine learning on Apple silicon”, Apple, Accessed: May 12, 2026. [Online]. Available: `https://github.com/ml-explore/mlx`.

[27] G. Gerganov et al. “llama.cpp: Port of facebook’s LLaMA model in c/C++”, Accessed: May 12, 2026. [Online]. Available: `https://github.com/ggerganov/llama.cpp`.

[28] Ollama. “Ollama: Get up and running with large language models locally”, Accessed: May 12, 2026. [Online]. Available: `https://ollama.com/`.

[29] HuggingFace. “PEFT: State-of-the-art parameter-efficient fine-tuning”, Accessed: May 12, 2026. [Online]. Available: `https://github.com/huggingface/peft`.

[30] T. Kočiský et al., “The NarrativeQA reading comprehension challenge”, *Transactions of the Association for Computational Linguistics (TACL)*, vol. 6, pp. 317–328, 2018.

[31] Z. Yang et al., “HotpotQA: A dataset for diverse, explainable multi-hop question answering”, in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2018.

[32] W. Fei et al., “Extending context window of large language models via semantic compression”, in *Findings of the Association for Computational Linguistics (ACL Findings)*, 2024.

[33] A. van den Oord, Y. Li, and O. Vinyals, *Representation learning with contrastive predictive coding*, arXiv preprint arXiv:1807.03748, InfoNCE., 2018.

[34] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, “A simple framework for contrastive learning of visual representations”, in *Proc. International Conference on Machine Learning (ICML)*, 2020.

[35] Cheng et al., *xRAG: Extreme context compression for retrieval-augmented generation with one token*, arXiv preprint arXiv:2405.13792, 2024.

[36] C. Packer et al., *MemGPT: Towards LLMs as operating systems*, arXiv preprint arXiv:2310.08560, 2023.

[37] Xu et al., “A-MEM: Agentic memory for LLM agents”, in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[38] Chhikara et al., *Mem0: Building production-ready AI agents with scalable long-term memory*, arXiv preprint arXiv:2504.19413, 2025.

[39] Rasmussen et al., *Zep: A temporal knowledge graph architecture for agent memory*, arXiv preprint arXiv:2501.13956, 2025.

[40] Yu et al., *MemAgent: Reshaping long-context LLM with multi-conv RL-based memory agent*, arXiv preprint arXiv:2507.02259, 2025.

[41] G. Li, H. A. A. K. Hammoud, H. Itani, D. Khizbullin, and B. Ghanem, *CAMEL: Communicative agents for “mind” exploration of large scale language model society*, arXiv preprint arXiv:2303.17760, 2023.

[42] Zhang et al., *AgentPrune: Cost-efficient multi-agent communication via graph sparsification*, arXiv preprint arXiv:2410.02506, 2024.

[43] Wang et al., *AgentDropout: Token-efficient multi-agent collaboration via dynamic agent removal*, arXiv preprint arXiv:2503.18891, 2025.

[44] Ye et al., “KVCOMM: Online cross-context KV-cache communication for efficient LLM inference”, in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[45] Haseeb, *Context engineering for multi-agent LLM code assistants using elicit, NotebookLM, ChatGPT, and Claude code*, arXiv preprint arXiv:2508.08322, 2025.

[46] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks”, in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[47] P. Sarthi, S. Abdullah, A. Tuli, S. Khanna, A. Goldie, and C. D. Manning, “RAPTOR: Recursive abstractive processing for tree-organized retrieval”, in *Proc. International Conference on Learning Representations (ICLR)*, 2024.

[48] D. Edge et al., *From local to global: A GraphRAG approach to query-focused summarization*, arXiv preprint arXiv:2404.16130, 2024.

[49] B. J. Gutiérrez, Y. Shu, Y. Gu, M. Yasunaga, and Y. Su, "HippoRAG: Neurobiologically inspired long-term memory for large language models", in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[50] B. J. Gutiérrez, Y. Shu, W. Qi, S. Zhou, and Y. Su, "From RAG to memory: Non-parametric continual learning for large language models", in *Proc. International Conference on Machine Learning (ICML)*, 2025.

[51] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection", in *Proc. International Conference on Learning Representations (ICLR)*, 2024.

[52] Anthropic. "Introducing contextual retrieval", Anthropic News, Accessed: May 12, 2026. [Online]. Available: `https://www.anthropic.com/news/contextual-retrieval`.

[53] N. F. Liu et al., "Lost in the middle: How language models use long contexts", *Transactions of the Association for Computational Linguistics (TACL)*, vol. 12, pp. 157–173, 2024.

[54] Y. Bai, S. Tu, J. Zhang, et al., *LongBench v2: Towards deeper understanding and reasoning on realistic long-context multitasks*, arXiv preprint arXiv:2412.15204, 2024.

[55] G. Mialon, C. Fourrier, C. Swift, T. Wolf, Y. LeCun, and T. Scialom, *GAIA: A benchmark for general AI assistants*, arXiv preprint arXiv:2311.12983, 2023.

[56] X. Liu, H. Yu, H. Zhang, Y. Xu, et al., "AgentBench: Evaluating LLMs as agents", in *Proc. International Conference on Learning Representations (ICLR)*, 2024.

[57] Zhou et al., *Privacy-Aware retrieval-augmented generation*, arXiv preprint arXiv:2503.15548, 2025.

[58] Bassit and Boddeti, *SecureRAG: Privacy-preserving retrieval-augmented generation*, arXiv preprint, 2025.

[59] Li et al., "SecurityLingua: Adversarial robustness for prompt compression", in *Proc. Conf. on Language Modeling (CoLM)*, 2025.

[60] L. Lovén, "Real-time AI service economy: A polymatroidal framework for service markets", *IEEE Transactions on Services Computing (under review)*, 2025, Internal FCG reference, "Paper 1".

[61] Saleh et al., "MemIndex: Agentic event-based distributed memory management for large language model agents", *ACM Transactions on Autonomous and Adaptive Systems (TAAS)*, 2025.

[62] A. Saleh, R. Morabito, S. Dustdar, S. Tarkoma, S. Pirttikangas, and L. Lovén, "Towards message brokers for generative AI: Survey, challenges, and opportunities", *ACM Computing Surveys*, vol. 58, no. 1, Article 20, 2025. DOI: `10.1145/3742891`.

[63] Meta AI, *The Llama 3 herd of models*, arXiv preprint arXiv:2407.21783, 2024.

[64] E. J. Hu et al., "LoRA: Low-rank adaptation of large language models", in *Proc. International Conference on Learning Representations (ICLR)*, 2022.

[65] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, "SQuAD: 100,000+ questions for machine comprehension of text", in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2016.

[66] C.-Y. Lin, “ROUGE: A package for automatic evaluation of summaries”, in *Text Summarization Branches Out*, 2004, pp. 74–81.

[67] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, “BERTScore: Evaluating text generation with BERT”, in *Proc. International Conference on Learning Representations (ICLR)*, 2020.

[68] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, *RAGAS: Automated evaluation of retrieval augmented generation*, arXiv preprint arXiv:2309.15217, 2023.

[69] Confident AI, *DeepEval: The LLM evaluation framework*, GitHub repository, 2024. Accessed: May 12, 2026. [Online]. Available: `https://github.com/confident-ai/deepeval`.

[70] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York: Chapman & Hall, 1993.

[71] H. B. Mann and D. R. Whitney, “On a test of whether one of two random variables is stochastically larger than the other”, *Annals of Mathematical Statistics*, vol. 18, no. 1, pp. 50–60, 1947.

[72] F. Wilcoxon, “Individual comparisons by ranking methods”, *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.

[73] S. Holm, “A simple sequentially rejective multiple test procedure”, *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.

[74] N. Cliff, “Dominance statistics: Ordinal analyses to answer ordinal questions”, *Psychological Bulletin*, vol. 114, no. 3, pp. 494–509, 1993.

7. APPENDICES

The appendices collect three artefacts that support the main text but would interrupt its narrative: the memory-bus HTTP contract (Appendix A), the canonical long-format results-dataframe schema used by every experiment runner (Appendix B), and an example C1 workload instance from family (a) showing the source-fragment structure (Appendix C). The full reproducibility package, including the model cards and the data cards, is shipped with the open-source release described in Section 5.5 of Chapter Chapter 5.

7.1. *

Appendix A. Memory-Bus HTTP Contract

The reference memory-bus service exposes four endpoints. The contract below summarises the request and response shapes; the authoritative OpenAPI schema lives at `docs/openapi.json` in the repository and is regenerated via `make bus-openapi`.

`POST /v1/write` Body: a `Fragment` JSON object (`fragment_id`, `text`, `tags`) and an optional `target_ratio`. Returns the assigned `slot_id`, the audit row identifier, and the hex-encoded `chain_hash`. HTTP 201 on success; 403 on policy denial.

`GET /v1/read/{slot_id}` Returns a `CompressedSlot` with its payload, tags, compressor identifier, nominal ratio, and creation timestamp. HTTP 200 on success; 403 on policy denial; 404 on slot not found.

`POST /v1/subscribe` Body: a `SubscribeRequest` with a `query` string, a `ttl_seconds` positive integer, and a top-*k* integer. Returns a server-sent-event stream emitting newly-matched `slot_id`s with their cosine scores until the TTL elapses.

`GET /v1/audit/{slot_id}` Returns the chronological list of `AuditRow` entries that mention the given slot.

Every request is processed by the `PolicyMiddleware` which parses the requester principal from the `X-M6-Principal` header in the development build and from an OAuth/JWT verifier in a production deployment. The principal carries a 64-bit access-control mask and a 5-tier classification level (PUBLIC < INTERNAL < CONFIDENTIAL < RESTRICTED < SECRET). A request is granted access to a slot when the principal's mask is a superset of the slot tag's mask and the principal's classification is at least the slot tag's classification.

7.2. *

Appendix B. Long-Format Results Schema

Every experiment runner writes its results to a CSV that conforms to the long-format schema implemented as `m6.evaluation.statistics.LongDF`. The columns are listed in Table 4.

Table 4: The canonical long-format dataframe schema used by every experiment runner. The same schema underlies every CSV under `results/`, which makes cross-hypothesis analysis a single `pandas.concat` away.

Column	Type	Notes
`experiment_id`	str	unique per run
`hypothesis`	str	one of `h1` . . . `h4`
`compressor`	str	e.g. `lingua2`, `filter`, `icae`, `icae-tag`, `none`
`ratio`	float	nominal compression ratio
`actual_ratio`	float	measured ratio post-compression
`pipeline`	str	one of `none`, `P1`, `P2`, `P3`
`model`	str	e.g. `llama-3.1-8b-instruct`
`model_size`	str	`7b`, `13b`
`workload_family`	str	`a`, `b`, or `c`
`workload_id`	str	e.g. `c1-a-007`
`seed`	int	
`metric`	str	e.g. `coord_success`, `qa_f1`, `preservation_rate`
`value`	float	
`wallclock_ms`	int	
`eur_cost`	float	
`run_id`	str	
`git_sha`	str	
`config_hash`	str	
`created_at`	str	ISO-8601 UTC
`invalid`	bool	control-condition variance dominates?
`invalid_reason`	str	free-text explanation if invalid

7.3. *

Appendix C. Example C1 Workload (family a)

The following JSON object illustrates a single instance of C1 family (a) (cross-document fact aggregation). Each fragment represents one institutional system; the planner is asked to aggregate hours and budget across all eight; the critic verifies the answer against the ground-truth aggregate.

```
{
  "workload_id": "c1-a-007",
  "family": "a",
  "seed": 1247,
  "tag_distribution": "skewed",
  "initial_prompt":
    "Aggregate the period-end report for project P-3219 across the
     following systems: publication_db, contract_db, moodle, patio,
     peppi, crm, tatu_sap, sap_travel. Report total hours and total
     budget.",
  "expected_answer": "hours=853;budget=148340",
  "n_agents": 8,
```

```
  "fragments": [
    {
      "fragment_id": "c1-a-007/publication_db",
      "text": "[publication_db] Project P-3219 - period 2025-Q3. ...",
      "tags": {"acl_mask": 12876, "classification": 1}
    },
    {
      "fragment_id": "c1-a-007/contract_db",
      "text": "[contract_db] Project P-3219 - period 2025-Q3. ...",
      "tags": {"acl_mask": 8901, "classification": 2}
    }
    // (six more fragments, one per system)
  ],
  "sub_tasks": [
    {
      "sub_task_id": "c1-a-007/sub-0",
      "description":
        "Extract recorded hours and budget for project P-3219 from
         publication_db.",
      "expected_solver": "worker-0",
      "expected_answer": "hours=112;budget=18420"
    }
    // (seven more sub-tasks)
  ],
  "protected_facts": [
    {
      "fragment_id": "c1-a-007/contract_db",
      "fact": "contract_db.budget=22150",
      "yesno_questions": [
        "Did contract_db for project P-3219 exceed EUR 27150 in budget?",
        "Was contract_db's recorded hours for P-3219 more than 142?"
      ],
      "answers": ["no", "no"]
    }
  ]
}
```

The full dataset is reproduced from `configs/benchmark/c1-v0.1.yaml` via `make bench-generate`; the manifest carries the configuration hash so any reader can verify they are looking at the same release.